

Aula 19 - Manipulação do DOM – II

Descrição: Exposição de *createElement*, *appendChild* e *removeChild*. Prática de renderização de listas dinâmicas a partir de um array. Criação de função genérica *renderList*(items, container) e atividade em grupo focada em hierarquia do DOM com a construção de uma tabela dinâmica de produtos.

Cenário Real: Pense na interface da Netflix, do Mercado Livre ou no feed do Instagram. Quando rolamos a página ou pesquisamos por "notebooks", os resultados aparecem na tela sem que a página inteira recarregue. Nos bastidores, a aplicação está recebendo dados (geralmente em formato JSON) e construindo o visual em tempo real.

Se os desenvolvedores dessas plataformas tivessem que escrever o HTML estático de cada produto manualmente, a manutenção seria impossível e a escalabilidade, nula. No mercado atual, dominado por Single Page Applications (SPAs) e consumo de APIs, a habilidade de ler um conjunto de dados brutos e transformá-los dinamicamente em elementos visuais na tela é o coração do desenvolvimento Front-end.

Hoje, deixamos de ser apenas "escritores de HTML" para nos tornarmos arquitetos de interfaces dinâmicas.

Pergunta:

"Se você precisasse exibir uma lista de 500 produtos em uma página, e fizesse isso escrevendo as tags `<li>` manualmente no HTML, qual seria o maior risco técnico e financeiro para a empresa caso a classe CSS de todos os itens ou a regra de desconto mudasse de repente?"

Criando e Inserindo Elementos (*createElement* e *appendChild*)

A. CONCEITO

Essa analogia da **Árvore de Família** explicada na aula anterior é uma das melhores formas de entender o que acontece nos bastidores do navegador. Para explicar como isso funciona na prática, vamos dividir o "nascimento" desse elemento em três etapas fundamentais. Imagine que estamos em um canteiro de obras digital:

1. O Nascimento no "Limbo" (*createElement*)

Quando você executa `document.createElement('li')`, o JavaScript reserva um espaço na memória RAM.

- **Na prática:** O objeto existe, ele tem propriedades, mas ele **não tem endereço**.
- **O estado:** Se você olhar para a página do navegador, nada mudou. O elemento está "flutuando" na memória, como um bebê que nasceu em outra cidade e ainda não foi registrado na árvore genealógica da família que mora no HTML.

2. A Configuração (Personalidade)

Antes de apresentar o novo filho à família, você precisa dizer quem ele é. É aqui que definimos o `textContent`, adicionamos classes (`classList.add`) ou definimos IDs.

JavaScript

```
const novoFilho = document.createElement('li'); // Nascimento
novoFilho.textContent = 'Item 1';                // Nome e Identidade
novoFilho.style.color = 'blue';                  // Aparência
```

Nesse ponto, o elemento já está "pronto", mas ainda é invisível para o usuário final porque ele não está conectado ao `<body>` (o avô).

3. A Adoção Oficial (`appendChild`)

Esta é a etapa crucial. O método `appendChild()` é o aperto de mão entre o que está na memória e o que aparece na tela.

- **O Pai:** Você seleciona um elemento que já está na árvore (ex: `ul id="lista"`).
- **A Ação:** Ao fazer `pai.appendChild(novoFilho)`, você está criando um **vínculo físico** (um ponteiro) na árvore do DOM.
- **O Resultado:** No exato milissegundo em que esse vínculo é criado, o motor de renderização do navegador (Reflow/Repaint) percebe a mudança e desenha o item na tela.

Por que isso é o "Padrão Ouro"? (Segurança)

- **innerHTML:** É como derrubar uma parede da casa e reconstruí-la inteira só para colocar um quadro novo. Além de lento, se houver um "vírus" (script malicioso) no texto, ele será executado.
- **createElement + appendChild:** Você está manipulando **objetos**, não textos. O navegador entende que `textContent` é apenas texto puro. Mesmo que alguém tente injetar um `<script>`, o navegador o tratará como simples palavras, protegendo sua aplicação de ataques XSS (Cross-Site Scripting).

Resumo do fluxo:

1. **Cria** (está na RAM)
2. **Configura** (define o que ele é)
3. **Anexa** (aparece na tela).

Ficou clara a diferença entre o elemento existir na memória e existir na página? O comando de criação acontece **exclusivamente no lado do cliente** (no navegador do usuário) e o momento exato depende de como você configurou o seu código JavaScript.

Para ser bem didático, vamos separar o "Onde" do "Quando":

Onde ele é enviado?

O comando não viaja para a internet. Ele é enviado para a **Engine de JavaScript** do navegador (como o V8 no Chrome ou o SpiderMonkey no Firefox).

- **O Palco:** Tudo acontece dentro da **memória RAM** do computador ou celular de quem está acessando o site.
- **O Processo:** Quando o navegador lê a linha `document.createElement()`, ele não pede permissão ao servidor para criar o elemento. Ele simplesmente reserva um "lote" na memória local e diz: "Reservado para um novo `<li>`".

Quando ele é enviado?

Existem três cenários comuns para esse "nascimento":

- **Ao carregar a página:** Você quer que a lista já comece com itens que não estavam no HTML original. O comando é disparado assim que o script é lido.
- **Interação do Usuário (O mais comum):** Quando alguém clica em um botão "Adicionar Carrinho" ou "Enviar Comentário". O comando de criação é enviado no exato momento do clique.
- **Resposta de uma API:** O seu site pergunta ao servidor: "Quais são as notícias de hoje?". O servidor responde com dados (JSON). O JavaScript recebe esses dados e, **só então**, envia o comando de criação para cada notícia recebida.

O Fluxo Visual da Execução

Imagine o console do navegador como o "centro de comando":

1. **Gatilho:** O usuário clica em um botão.
2. **Comando de Criação:** O JS envia `createElement` para a RAM.

3. **Comando de Configuração:** O JS envia as propriedades (nome, cor).
4. **Comando de Anexação:** O JS envia o `appendChild` para a árvore do DOM.

Por que isso é importante?

Se o comando fosse enviado para o servidor toda vez que você criasse um elemento simples, a internet seria incrivelmente lenta. Como ele é enviado apenas para a memória local, o processo de criar um elemento e exibi-lo leva, geralmente, menos de **1 milissegundo**.

Faz sentido pensar que o JavaScript é como um "mestre de obras" que já está dentro da casa (o navegador), pronto para agir sem precisar ligar para o arquiteto (o servidor) para cada tijolo novo?

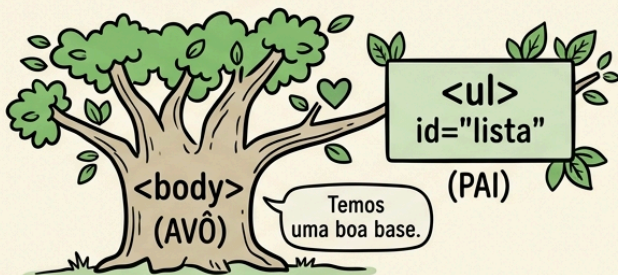
B. VISUALIZANDO

Imagine uma árvore de família.

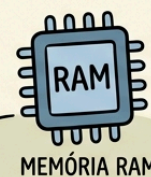
1. A árvore original tem um avô (`<body>`) e um pai (`<ul id="lista">`).
2. O `createElement('li')` "nasce" como um filho em outra cidade (na memória RAM), sem conexão com a família.
3. Configuramos esse filho dando-lhe um nome (`textContent = 'Item 1'`).
4. O `appendChild(filho)` atua como a certidão de adoção: ele pega a tag `<ul>` e diz: "este `<li>` agora é seu último filho". A partir desse milissegundo, o item aparece na tela do usuário.

UMA ÁRVORE DE FAMÍLIA DO DOM

1 A ÁRVORE ORIGINAL



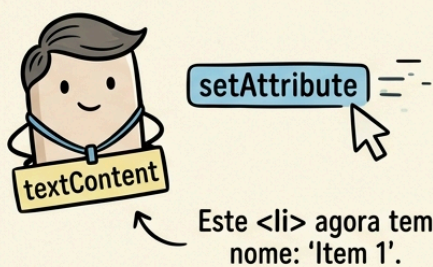
2 createElement('li') "NASCE" EM OUTRA CIDADE (NA MEMÓRIA RAM)



3 CONFIGURAMOS ESSE FILHO (NOMEANDO)



3 CONFIGURAMOS ESSE FILHO (NOMEANDO)



4 appendChild(filho) É A ADOÇÃO & APARECE NA TELA



C. EXEMPLO DE CÓDIGO

JavaScript

```
// 1. Seleccionamos o elemento pai que já existe no HTML
```

```
const listaProdutos = document.querySelector('#lista-produtos');
```

```
// 2. Criamos um novo nó do tipo Element (uma tag <li> vazia na memória)
```

```
const novoProduto = document.createElement('li');
```

```
// 3. Adicionamos conteúdo e classes de estilo ao novo nó (Segurança: textContent em vez de innerHTML)
```

```
novoProduto.textContent = 'Monitor UltraWide 29'';
```

```
novoProduto.classList.add('item-produto');
```

```
// 4. Conectamos o novo nó à árvore do DOM (inserindo no final da lista)
```

```
listaProdutos.appendChild(novoProduto);
```

D. EXPLICAÇÃO DO CÓDIGO

- Primeiro, capturamos o contêiner `<ul>` através do seu ID.
- Na linha 2, chamamos o "método de fábrica" `createElement` passando a string `'li'`. Note que a tag não leva os sinais de menor/maior (`<` `>`).
- Na linha 3, populamos o conteúdo do `<li>` usando `textContent`. Optamos por ele no lugar do `innerHTML` porque ele trata o texto de forma literal, garantindo que se houvesse alguma tag maliciosa (ex: `<script>`), ela seria lida como texto puro.
- Finalmente, o `appendChild` anexa o nosso `li` perfeitamente configurado dentro do `ul`. Se não fizéssemos a linha 4, o monitor nunca apareceria na tela.

E. ATENÇÃO / DICA / REFLITA

- **Dica:** O uso de IA Generativa (como o ChatGPT ou Gemini) pode acelerar a escrita de templates de inserção, mas atenção: IAs costumam sugerir o uso de `innerHTML += "<li>...</li>"` por ser mais curto. Isso é uma **péssima prática** de mercado, pois **recria todo o DOM** interno a cada iteração, destruindo event listeners antigos e abrindo brechas de segurança (XSS). Use sempre a criação de nós (`createElement`).

F. EXERCÍCIOS RÁPIDOS

1. O que acontece com um elemento criado via `createElement` antes de usarmos `appendChild`?
2. Por que o `textContent` é mais seguro que o `innerHTML` ao inserir dados desconhecidos?
3. Escreva 2 linhas de código para criar um botão (`<button>`) e colocar o texto "Enviar" dentro dele.
4. O código `document.createElement('DIV')` (em maiúsculas) funciona? Teste e explique.
5. Se você fizer `appendChild` de um elemento que já está na tela para outro contêiner, o que acontece? Ele duplica ou move? (Dica: tente no console).

Removendo Elementos da Árvore (`removeChild` e `remove`)

A. CONCEITO

Essa é a conclusão lógica do ciclo de vida que começamos a estudar. Se o `appendChild` é a "adoção", o `remove` é a "emancipação" ou a saída estratégica de cena.

Aqui está a explicação de como isso funciona na prática:

1. O Conceito de Desconexão

Quando você remove um elemento, você está cortando os fios que o ligam à **Árvore do DOM**.

- **Na Interface (UI):** O elemento desaparece instantaneamente. O navegador recalcula o espaço (se você remover um item do meio de uma lista, os itens de baixo "sobem" para ocupar o buraco).
- **Nos Bastidores (RAM):** O elemento não é destruído imediatamente pelo computador. Ele volta para aquele "limbo" da memória onde nasceu. Se você tiver uma variável guardando ele, você pode até "readotá-lo" em outro lugar. Se não houver ninguém segurando ele, o **Garbage Collector** (Lixeiro do Navegador) o apaga de vez.

2. A Evolução do Comando: Do Indireto ao Direto

O Velho Jeito: `removeChild()`

Imagine que você quer expulsar um convidado de uma festa, mas, por uma regra estranha, você não pode falar com ele; você precisa achar o **dono da casa** e pedir para ele expulsar a pessoa.

- **Código:** `pai.removeChild(filho)`
- **Problema:** Às vezes, o seu código sabe qual é o botão que foi clicado, mas não sabe quem é o "pai" dele. Você teria que fazer algo como `botao.parentNode.removeChild(botao)`. Meio barulhento, né?

O Jeito Moderno: `remove()`

É a autonomia total. O elemento recebe a ordem e ele mesmo se retira da árvore.

- **Código:** `elemento.remove()`
- **Vantagem:** É mais limpo, fácil de ler e direto ao ponto. É o padrão atual para quase todas as situações.

3. Por que isso é vital? (Gerenciamento de Memória)

Já mencionamos os **Memory Leaks** (Vazamentos de Memória). Imagine que seu site é uma lista de tarefas. Se você "deleta" 100 tarefas da tela usando truques visuais (como esconder com CSS), mas não as remove do DOM, o navegador continua gastando energia e memória para processar esses itens invisíveis. Remover o nó garante que o navegador pare de se preocupar com aquele pedaço de código, mantendo o site leve e rápido, especialmente em celulares.

Exemplo Prático: O Botão "Matador"

Uma das melhores formas de ver isso funcionando é criar um elemento que se deleta ao ser clicado:

JavaScript

```
const item = document.createElement('li');
item.textContent = 'Clique em mim para eu sumir';
```

// Ao clicar, o próprio elemento executa o comando de saída

```
item.addEventListener('click', () => {
  item.remove();
});
```

```
document.querySelector('ul').appendChild(item);
```

Resumo da "Morte" de um Elemento:

1. **Gatilho:** Uma ação ocorre (clique no "X" de fechar).
2. **Comando:** O JS executa `.remove()`.
3. **Reflow:** O navegador redesenha a página sem aquele item.
4. **Limpeza:** Se não houver referências, a memória RAM é liberada.

Faz sentido como o ciclo se fecha? Agora você sabe criar, configurar, dar um endereço e, quando necessário, dar o "**cartão vermelho**" para um elemento!

B. ATENÇÃO / DICA / REFLITA

- **Atenção!** Remover um elemento da tela não o deletará da memória imediatamente se você ainda tiver uma variável apontando para ele no seu script. Isso cria vazamento de memória. Se não for usar o elemento novamente, certifique-se de que não há variáveis "segurando" sua referência.

C. EXERCÍCIOS RÁPIDOS

1. Qual a diferença conceitual entre `removeChild` e `remove`?
2. Se executarmos `pai.removeChild(filho)` e o "filho" não pertencer àquele "pai", o que ocorre no console?
3. Crie um script que selecione o primeiro `<li>` de uma página e o destrua usando `.remove()`.

4. É possível guardar um elemento removido em uma variável e inseri-lo de volta em outro lugar depois?
 5. Se uma tabela inteira (<tbody>) for removida, o que acontece com os <tr> e <td> dentro dela?
-

Renderização Dinâmica e Clean Code: A Função renderList

A. CONCEITO

Essa é a transição do "**codificador amador**" para o **Engenheiro de Software**. O conceito aqui é transformar uma tarefa manual e repetitiva em uma **máquina de automação**.

Imagine que você é um carpinteiro. Em vez de fabricar uma cadeira do zero toda vez (medir, cortar, lixar, montar), você constrói um **molde (a função)**. Agora, sempre que alguém quer uma cadeira, você só joga a madeira no molde. Veja como isso funciona na prática, dividindo em três pilares:

1. O Problema: Código "Hardcoded" (Engessado)

Quando você escreve um código preso a um ID específico (como `#lista-produtos`), você criou um especialista que só sabe fazer uma coisa. Se amanhã você precisar listar "**Clientes**", esse código é inútil. Você teria que copiar e colar, mudando apenas o nome das variáveis.

- **Consequência:** Se você descobrir um erro na forma como cria os itens, terá que consertar em 10 lugares diferentes. Isso é a **Dívida Técnica**.

2. A Solução: Desacoplamento e Abstração

A função `renderList` age como um **serviço de entrega**. Ela não quer saber **o que** está dentro da caixa (**os dados**) nem **onde** fica a casa (**o contêiner**). Ela apenas sabe como realizar a entrega.

Os Parâmetros como "Contratos":

- **Dados (Array):** É o combustível. Pode ser uma lista de preços, nomes de frutas ou e-mails.
- **Contêiner (Pai):** É o endereço de destino. Pode ser uma `<ul>`, uma `<div>` ou um `<section>`.

3. Na Prática: Antes vs. Depois

O jeito repetitivo (Amador):

JavaScript

// Código preso ao produto

```
produtos.forEach(p => {  
  const li = document.createElement('li');  
  li.textContent = p.nome;  
  document.getElementById('lista-produtos').appendChild(li);  
});
```

O jeito Reutilizável (Engenharia):

JavaScript

```
function renderList(dados, containerPai) {  
  // Limpamos o pai para evitar duplicatas (Reset)  
  containerPai.innerHTML = "";  
  
  dados.forEach(item => {  
    const li = document.createElement('li');  
    li.textContent = item;  
    containerPai.appendChild(li);  
  });  
}
```

```
// Agora a mesma função serve para TUDO:  
renderList(meusProdutos, listaDeProdutosDOM);  
renderList(vendedores, listaDeVendedoresDOM);
```

Por que isso é o "Padrão Ouro"?

1. **Manutenção Simples:** Quer adicionar uma classe de estilo em todos os itens do site? Você muda apenas **uma linha** dentro da função `renderList`.
2. **Escalabilidade:** Seu site pode crescer de 1 para 100 listas diferentes sem aumentar o tamanho do seu arquivo JavaScript significativamente.
3. **Testabilidade:** É muito mais fácil testar se uma função genérica funciona do que testar 10 blocos de código espalhados.

Em resumo: Você deixa de escrever **o que fazer** em cada detalhe e passa a escrever **como as coisas devem ser feitas** de forma geral. Ficou claro como essa "fábrica de listas" economiza o seu tempo e o processamento do navegador?

B. VISUALIZANDO

Fluxo do Código Acoplado (Ruim):

Array de Produtos -> Loop Específico -> Conectado a fogo em `#lista-produtos`. (Se pedir outra lista, tem que copiar/colar tudo).

Fluxo do Código Reutilizável (Bom):

Array X + Contêiner Y -> `renderList(data, container)` -> Injeta na Tela.

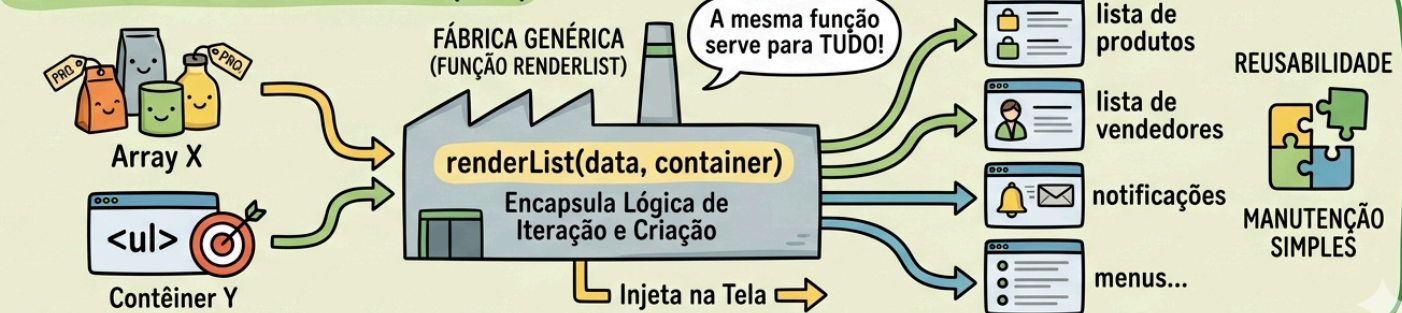
(A mesma função serve para produtos, vendedores, notificações, menus...).

CONCEITOS DE ENGENHARIA DE SOFTWARE: ACOPLAMENTO VS. REUSABILIDADE

FLUXO DO CÓDIGO ACOPLADO (RUIM)



FLUXO DO CÓDIGO REUTILIZÁVEL (BOM)



C. ATENÇÃO / DICA / REFLITA

- **Reflita:** Soft Skill de **Comunicação Técnica**. Como você explicaria para o seu Gerente de Projeto que atrasou a entrega em 1 hora porque precisou "**refatorar**" a função de listar elementos? A justificativa correta não é "**porque o código está feio**", mas sim "**para diminuir o tempo de entrega** das próximas 10 listas do projeto e evitar bugs de inconsistência visual".

D. EXERCÍCIOS RÁPIDOS

1. Por que é importante limpar o container (ex: `container.innerHTML = ''`) no início da função `renderList`?
2. O que acontece se passarmos uma string em vez de um Array para o parâmetro `items` no `.forEach`?
3. Modifique a função acima para adicionar a classe CSS "list-item" a cada `<li>` criado.
4. Se tivéssemos um array `['Alice', 'Bob']` e um `<ul id="users">`, como você chamaria a função `renderList`?
5. Qual princípio de Engenharia de Software estamos aplicando ao evitar a duplicação do `forEach`?

MÃO NA MASSA (utilizar o arquivo EXEaula19.html)

Para entender o código completo do nosso **Sistema de Inventário**, precisamos olhar para ele com a mentalidade de um Engenheiro de Software. Na nossa aula, vimos que manipular o DOM não é apenas sobre jogar elementos na tela, mas sim sobre **sincronizar dados com a interface**.

Vamos desconstruir o código passo a passo, conectando tudo o que aprendemos hoje: Mock Data, criação de elementos (`createElement`), aninhamento (`appendChild`) e deleção.

O Cérebro do Sistema (O "Estado")

No topo do nosso script, temos a variável mais importante:

JavaScript

```
let dadosProdutos = [];
```

Este Array é a "fonte da verdade". Na vida real, ele começaria preenchido com dados vindos de um Banco de Dados. Em nossa aplicação, tudo o que o usuário vê na tela é apenas um reflexo visual (um "espelho") do que está dentro desse Array.

A Fábrica de Renderização (`renderizarTabela`)

Essa é a nossa função genérica. Ela é responsável por ler a "fonte da verdade" e traduzi-la em elementos visuais.

- **A Limpeza (`innerHTML = ''`):** A primeira coisa que a fábrica faz é varrer a tela. Se não fizermos isso, toda vez que adicionarmos um produto, a tabela inteira seria duplicada abaixo da antiga.
- **A Iteração (`forEach`):** A fábrica percorre o Array item por item.
- **A Construção (`createElement` e `innerHTML`):** Para cada item, criamos o "Pai" (a linha `<tr>`). Em vez de fazer um `createElement` para cada célulazinha de texto, usamos uma técnica mista: injetamos o HTML simples via Template Literals (```) para ganhar tempo.
- **O Botão Dinâmico:** É aqui que a mágica acontece. O botão de "Remover" não existe no HTML original; ele é forjado na hora:

JavaScript

```
const btnDeletar = document.createElement('button');  
btnDeletar.onclick = () => { ... }
```

- **A Adoção (`appendChild`):** Por fim, o botão é colocado dentro do `<td>`, o `<td>` é colocado dentro do `<tr>`, e o `<tr>` é anexado ao `<tbody>`. A hierarquia complexa está montada!

O Fluxo de Entrada (`adicionarProduto`)

Quando o usuário clica em "**Salvar Produto**", o código não cria o elemento na tela imediatamente. Ele segue um fluxo de trabalho rigoroso:

1. **Captura e Validação:** Ele lê os valores do `<input>` e verifica se o usuário não digitou bobeira (como um nome vazio).
2. **Atualização do Estado:** Ele empacota esses dados em um novo "Objeto JSON" e faz um `.push()` (empurra) para dentro do nosso Array `dadosProdutos`.
3. **Limpeza Visual:** Ele apaga o texto das caixas de input para o usuário poder digitar o próximo.
4. **O Gatilho:** Ele chama a `renderizarTabela()`. A fábrica liga as máquinas novamente, lê o Array (que agora tem um item a mais) e desenha a tabela nova.

O Fluxo de Saída (Remoção)

Na aula, vimos o `removeChild` e o `.remove()`. Mas, em um sistema guiado por dados (Data-Driven), nós não apagamos o elemento HTML diretamente. Nós apagamos o **dado**!

Quando o usuário clica no botão "Remover" forjado pela nossa fábrica, acontece o seguinte:

JavaScript

```
dadosProdutos.splice(index, 1);  
renderizarTabela();
```

1. O `.splice()` vai na nossa "fonte da verdade" (o Array) e arranca o objeto de lá.
2. Em seguida, chamamos a `renderizarTabela()`. Como a fábrica sempre limpa a tela e desenha apenas o que está no Array, o produto removido simplesmente não é desenhado na nova rodada. O resultado visual é que ele "sumiu".

Resumo da Ópera: O HTML fornece a casca. O JavaScript fornece a inteligência. Nós nunca mexemos na interface diretamente quando queremos adicionar ou deletar; nós alteramos o **Array** e pedimos para a função de renderização atualizar a tela. Esse é o coração da manipulação moderna do DOM!

Dica de Ouro para o Grupo:

Abram o **Console do Desenvolvedor (F12)**, vão na aba **Elements** e cliquem nas setinhas para expandir o `<tbody>`. Vocês verão exatamente a estrutura que o JavaScript "fabricou". Se deletarem um item do Array e chamarem a função novamente, verão a mágica da atualização dinâmica acontecer!

FAQ

Pergunta: Qual a diferença entre `appendChild` e `append`? Qual devo usar?

Resposta: O `appendChild` insere apenas um único objeto do tipo Node (nó) e retorna o próprio nó inserido. O `append` (mais moderno) aceita múltiplos nós ou até mesmo strings diretas (ex: `pai.append(div, "Texto solto")`), mas não tem valor de retorno. Em ambientes puristas e com grande exigência de compatibilidade e controle rigoroso, `appendChild` é mais seguro. Para agilidade moderna, `append` é amplamente utilizado.

Pergunta: Fazer muitos `appendChild` dentro de um loop de 10.000 itens trava a página?

Resposta: Sim! Toda vez que você altera o DOM visível, o navegador aciona um processo chamado Reflow e Repaint, o que é muito custoso em processamento. Para listas gigantes, usamos uma técnica avançada chamada DocumentFragment — uma "caixa invisível" onde anexamos os milhares de itens na memória primeiro e, no final, anexamos a caixa inteira ao DOM (1 único Reflow).

Pergunta: Posso usar `innerHTML` com crases (Template Literals) para montar a tabela? Não é mais fácil?

Resposta: É mais rápido de digitar, mas péssimo para a qualidade de software. O `innerHTML` destrói todo o contêiner para recriá-lo como texto. Isso mata eventos de clique (`addEventListener`)

previamente ligados, causa recarregamento de imagens aninhadas e abre as portas do seu sistema para Cross-Site Scripting (XSS), a principal falha de segurança na web. Prefira sempre criar os nós.

Pergunta: Se eu mudar um valor no meu Array de dados, a tabela atualiza sozinha?

Resposta: No Vanilla JS, não. O array e o DOM são entidades separadas. Após alterar o dado no Array (ex: `.push({novoDado})`), você é obrigado a chamar a função `renderList` ou `renderizarTabela` novamente para sincronizar a tela com os dados atualizados. Bibliotecas como React nasceram justamente para automatizar essa reatividade.

Pergunta: Meu `createElement` está gerando a tag na tela, mas sem conteúdo, mesmo eu colocando `node.value = 'Texto'`. Por quê?

Resposta: Depende da tag. Elementos estruturais (`div`, `p`, `li`, `td`) usam `textContent` (ou `innerText`) para o texto entre a abertura e fechamento da tag. Já elementos de formulário (como um `<input>`) usam a propriedade `.value`.

RESUMINDO

Para garantir que nosso código tenha qualidade e manutenibilidade, fechamos esta aula com o seguinte checklist:

- **Nó (Node):** Compreensão de que todo elemento em tela é um objeto tratado na memória do navegador.
 - **createElement:** A fábrica do JavaScript puro para inicializar elementos de marcação na memória antes de exibí-los.
 - **appendChild / remove:** A conexão e desconexão vitalícia dos elementos com a árvore do DOM visível.
 - **Segurança vs Rapidez:** Adoção do `textContent` para inserção de dados sobre o vulnerável `innerHTML`.
 - **Reusabilidade (Desacoplamento):** O conceito de criar funções genéricas (`renderList`) que dependem de parâmetros (dados e contêineres), eliminando repetição de código e facilitando manutenções futuras.
 - **Hierarquia Estrita:** O respeito à arquitetura Pai-Filho do HTML ao montar componentes complexos via JS (como Tabela -> Linha -> Célula).
-

EXERCÍCIOS DE FIXAÇÃO

Exercício 1 — Notificação Simples

Contexto: O usuário acabou de adicionar um produto no nosso sistema de gestão. Precisamos dar um feedback visual rápido para ele.

Missão: Crie um script dinâmico dentro da função `adicionarProduto()` que forja um elemento `<div>`, aplica uma classe CSS `.alert-success`, adiciona o texto "Produto cadastrado com sucesso!" usando a propriedade correta e anexa ao contêiner principal da página usando `appendChild`.

Entrega: Trecho de código JavaScript funcional.

Exercício 2 — O Gerente Mandou Apagar

Contexto: Se o usuário digitar um preço inválido, nosso sistema mostra um alerta de erro na tela com o id `#erro-validacao`. Quando ele corrigir, o erro deve sumir.

Missão: Selecione este elemento `#erro-validacao` via JS e elimine-o definitivamente do DOM utilizando a API nativa. Se certifique de fazer a validação para que não dê erro se o alerta já não estiver lá.

Entrega: Código JS (Máximo de 3 linhas).

Exercício 3 — Categorias Dinâmicas

Contexto: Nosso backend em PHP enviou um JSON com as categorias dos produtos para colocarmos no formulário de cadastro.

Missão: Dado o array `const categorias = [{id: 1, nome: 'Eletrônicos'}, {id: 2, nome: 'Móveis'}]`, crie uma função genérica que recebe essas categorias e itera sobre elas construindo tags `<option>`, configurando as propriedades `value` (com o ID) e `textContent` (com o nome). Em seguida, as anexe dentro de um `<select id="categoria">`.

Entrega: Função de iteração completa criadora de múltiplos atributos DOM.

Exercício 4 — Refatoração de Acoplamento

Contexto: Seu colega Dev Júnior tentou adicionar um rodapé com o total de itens no final da tabela e fez uma função engessada: `function gerarRodape() { document.querySelector('.container').innerHTML += "<div class='rodape'>Total atualizado</div>" }`.

Missão: Refatore o código desse colega. Remova o acoplamento e, principalmente, remova a falha de injeção trocando `innerHTML +=` por criação de nós nativa (`createElement`).

Entrega: Função refatorada com boas práticas (Clean Code).

Exercício 5 — Adicionar e Sincronizar (Limpar Estoque)

Contexto: O cliente pediu um botão de pânico para zerar o inventário inteiro de uma vez.

Missão: Programe uma função `limparEstoque()` que esvazia completamente o array `dadosProdutos` (nosso estado) e IMEDIATAMENTE chama a sua função `renderizarTabela()` feita anteriormente para atualizar a interface.

Entrega: Lógica que demonstra o Gerenciamento de Estado simples no JS.

Exercício 6 — Ações na Tabela Dinâmica

Contexto: A tabela de produtos agradou o cliente, mas agora ele quer uma nova ação direto por lá.

Missão: Modifique a função `renderizarTabela`. Ao lado do botão "Remover" que já existe na quarta coluna, crie um novo botão (`button`). Coloque o texto "Editar" neste botão e anexe-o na mesma célula (`td`).

Entrega: A nova função `renderizarTabela` incluindo o novo nó sendo processado hierarquicamente.

Exercício 7 — Fechamento por Escopo (Closure) no Botão

Contexto: Continuação do exercício anterior. Precisamos garantir que o sistema avise qual produto está sendo manipulado.

Missão: Adicione um evento de clique (`addEventListener` ou `.onclick`) ao botão "Editar" que você gerou lá dentro do loop do `forEach`. Quando clicado, o botão deve capturar o escopo e dar um `alert()` exibindo exatamente o `produto.nome` daquela respectiva linha mãe.

Entrega: O bloco de código que combina a criação do nó com a anexação imediata de um evento nele antes de ir pro DOM.

Exercício 8 — Dropdown Dependente

Contexto: Seleção de Categoria e Subcategoria no formulário de cadastro.

Missão: Você tem um objeto aninhado: `const subcategorias = { "Eletrônicos": ["Notebook", "Mouse"], "Móveis": ["Cadeira Gamer"] }`. Crie uma função que lê a chave selecionada no input de categoria e popula dinamicamente (usando nó `<option>`) a tag `<select id="subcategorias">` apenas com os itens pertinentes àquela categoria, garantindo que você zere/limpe a lista antes de adicionar as novas subcategorias.

Entrega: Lógica complexa combinando busca em objeto aninhado e re-renderização de DOM.

Exercício 9 — Cards de Portfólio (Visualização em Grade)

Contexto: O cliente quer um botão para alternar a visualização da Tabela para um formato de "Catálogo Visual".

Missão: Dado o nosso array `dadosProdutos`, crie dinamicamente uma estrutura mais profunda que a tabela. Você deve montar: `<div class='card-produto'>` contendo um `<h3>Nome do Produto</h3>` que é irmão de um `<p>Preço</p>`. Inserir tudo em uma `<section id='galeria'>`.

Entrega: Script rigoroso demonstrando a habilidade de montar ramificações laterais sem errar de pai.

Exercício 10 — Depuração com Console

Contexto: Um estagiário sugeriu que, na função `renderizarTabela`, ao invés de usar `appendChild`, o código deveria fazer `tbody.innerHTML += "<tr>...</tr>"` dentro do `forEach` para poupar linhas.

Missão: Explique do ponto de vista de Engenharia de Software por que remontar a tabela dessa forma abrupta a cada iteração destrói o conceito de sincronização, causa problemas de performance e por que é uma péssima prática de mercado.

Entrega: Um parágrafo argumentativo de alto nível técnico (Soft Skill de Code Review).

FECHAMENTO

Conexão com a Situação de Aprendizagem (SA):

Esta aula é o esqueleto da interface do nosso projeto final. O sistema de CRUD (Criar, Ler, Atualizar e Deletar) que vocês vão desenvolver precisará listar os registros do banco de dados na tela, como a listagem de usuários e serviços. Em breve, a Linguagem PHP trará esses dados do banco para o Frontend, e a função genérica `renderizarTabela` e as iterações de manipulação do DOM que escrevemos hoje serão responsáveis por apresentar toda essa informação ao cliente final de forma responsiva, dinâmica e sem gargalos de performance. Dominar esse conceito evita código repetido e garante requisitos de qualidade de software na auditoria final (Black-box).

Próxima Aula:

A tabela que fizemos hoje está muito bonita, mas é "muda". Nós inserimos dados na tela, mas como sabemos quando o usuário interage com eles? Na próxima aula, vamos dar vida e interatividade a essa estrutura avançando no poder do JavaScript Client-Side: o mapeamento de Eventos (Clicks, Teclado, Formulários).

Antes de vir para a próxima aula, pense e reflita: como o navegador sabe exatamente em qual botão "X" você clicou, se você gerou os 50 botões dinamicamente no mesmo milissegundo?